Tribhuvan University

Faculty of Humanities and Social Science

Supervisor Ai Agent

Submitted to

Department of Computer Application

Birat Kshitiz College

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by

Your name **6-2-21-09-2021**

July 2026

Under Supervision of

Supervisor name

Tribhuvan University

Faculty of Humanities and Social Science

Birat Kshitiz College

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by Dipesh Ray entitled "**Supervisor AI Agent**" in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

Supervisor name

SUPERVISOR

Department of Computer Application

Birat Kshitiz College

Tribhuvan University

Faculty of Humanities and Social Science

College Name

LETTER OF APPROVAL

This is to certify that this project prepared by Your name entitled "Supervisor ai agent" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

____________________ **Supervisor name** Department of Computer Application Birat Kshitiz College Biratnagar-2, Nepal	____________________ **Coordinator name** BCA Coordinator Department of Computer Application Birat Kshitiz College Biratnagar-2, Nepal
____________________ **Supervisior name** **Internal Examiner**	(External Examiner)

ABSTRACT

This project presents the design and implementation of **Supervisor AI Agent.** An intelligent web-based system that automatically checks and gives feedback on BCA project reports according to Tribhuvan University guidelines. The system aims to solve the problems of manual supervision, formatting mistakes, and difficulty in following strict rules for 4th, 6th, and 8th semester projects.

The system was developed using Node.js with TypeScript and Fastify on the backend, React.js with TypeScript on the frontend, and PostgreSQL with Prisma ORM. A hybrid approach was used that combines a **rule-based engine** for strict guideline checking and **local AI (Ollama)** for intelligent content feedback. Students can upload PDF or DOCX files, select their semester, and receive instant feedback with scores, missing sections, formatting issues, and suggestions.

The completed system successfully validates report structure, formatting standards, and provides useful supervisory feedback. This demonstrates its potential to help students improve their project reports and reduce workload for supervisors in the BCA program.

Keywords: *Supervisor AI Agent, TU BCA Guidelines, Rule-based Engine, Ollama, Project Report Validation, Hybrid AI System, PDF/DOCX Analysis*

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude and appreciation to all those who have supported and guided us throughout the development of the supervisor ai agent.

First and foremost, we would like to extend our deepest thanks to our project supervisor, **Basanta Chapagain** and our coordinator **Suresh Thapa**, for their invaluable guidance, expertise, and constant support. Their mentorship and constructive feedback have been instrumental in shaping the project and pushing us to deliver our best.

Lastly, we would like to acknowledge the countless individuals who have shared their knowledge and expertise, either through discussions, feedback, or research materials. Your contributions have greatly enriched our understanding and helped shape the final outcome of this project.

Sincerely,

Dipesh Ray

LIST OF ABBREVIATIONS

API	Application Programming Interface
DFD	Data Flow Diagram
ERD	Entity Relation Diagram
UI	User Interface
UX	User experience
PDF	Portable Document Format
AI	Artificial Intelligence
TU	Tribhuvan University
BCA	BCA of Computer Application
RAG	Retrieval Augmented Generation
ORM	Object Relational Mapping
JSON	JavaScript Object Notation
VS Code	Visual Studio Code

Table of Contents

List of Figure

List of Tables

CHAPTER 1
INTRODUCTION

1.1 Introduction

Preparing a final year project report is one of the most crucial yet stressful parts of the BCA program at Tribhuvan University. Every student must follow strict guidelines regarding structure, formatting, chapter organization, and referencing style. However, these guidelines are provided only in PDF format, which makes them difficult to understand and apply correctly. As a final year student working on my own project, I spent countless hours manually checking whether my report had all the required sections, proper margins, correct font styles, and appropriate chapter flow. Even after multiple revisions, I was never fully confident that my report met all the standards.

This struggle is not unique to me. Most of my classmates faced similar problems, often resulting in lost marks during internal evaluation or repeated corrections before the final defense. Many students end up submitting reports with missing preliminary pages, weak abstracts, improper referencing, or incorrect chapter structures. Realizing this common pain point, I decided to build Supervisor AI Agent, an intelligent web-based application that acts as a virtual project supervisor. Students can simply upload their project report in DOCX format, and the system automatically analyzes it against official TU BCA guidelines for 4th, 6th, and 8th semesters.

The application uses a powerful combination of a custom rule-based engine and AI technology (Ollama local LLM and Gemini API) to provide instant, detailed feedback. It not only detects missing sections and formatting issues but also gives smart suggestions to improve content quality. This project helped me turn my personal frustration into a practical solution that can benefit all BCA students preparing for their project submission and viva.

1.2 Problem Statement

While working on my project and helping friends with their reports, I observed the following common problems:

- Students frequently miss required sections like Supervisor's Recommendation, letter of Approval, List of figures, etc.

- Chapter structure and content flow often do not match the prescribed format for their semester.
- Formatting mistakes (fonts, margins, headings, page numbering) are very common.
- Abstract and reference are usually weak or incomplete.
- There is no easy way to get quick, reliable feedback before final submission

1.3 Objectives

The objectives of this project are follows as:

- To implement a robust rule-based engine that checks structure, required section, chapter and formatting.
- To integrate AI (Ollama + Gemini API) for intelligent content analysis and improvement suggestions
- To develop a user-friendly web application that generates detailed compliance reports with severity classifications (Critical. Major, Minor) and supports multiple semesters (4^{th}, 6^{th}, 8^{th}) based on semester-specific guideline rules.

1.4 Scope and Limitation

1.4.1 Scopes

- Docx/pdf file upload and text extraction.
- Semester-wise validation (4th, 6th, 8th semester).
- Detection of proposal vs final report automatically
- Rule-based checking + AI feedback generation.
- User dashboard with submission history and detailed reports.
- Support for common formatting standards (Times New Roman, margins, headings, etc.).

1.4.2 Limitations

- AI feedback quality depends on the LLM model used.
- Advanced features like real-time collaborative editing are not implemented.
- Plagiarism checking is not included in the current version.
- Limited to TU BCA guidelines only.

1.4 Development Methodology

For the development of Supervisor ai agent, I adopted the Agile Methodology using the Scrum framework. Since the project involved complex feature like rule-based validation, document processing and AI integration, Agile provided the necessary flexibility to adopt as I gained deeper understanding of TU guidelines and technical challenges. I divided the entire development into short sprints of 1-2 weeks, where each sprint focused on delivering a functional module such as user authentication, file upload, rule engine, AI feedback system, or frontend dashboard. At the end of every sprint, I tested the features, gathered feedback from my supervisor and classmates, and improvements in the next iteration. This iterative approach helped me identify and resolve issued early, resulting in better project quality and effective time management.

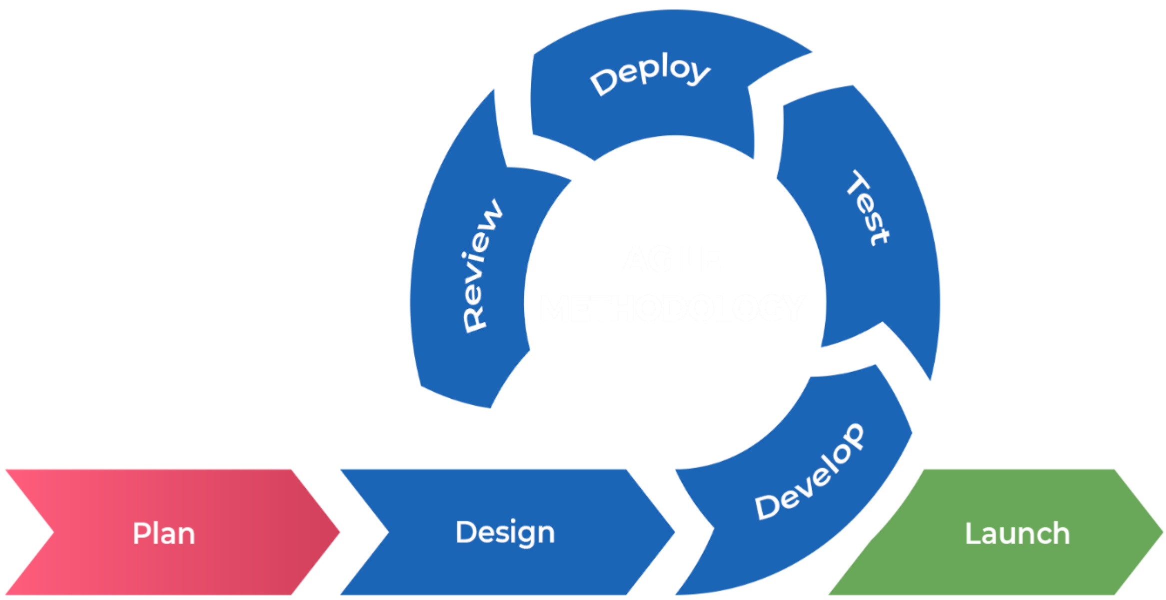


Figure 1: Agile Methodology for Supervisor AI Agent

1.5 Report Organization

Chapter 1: Introduction

It includes the general overview of the system and project as a whole. It includes the problem statement, Objective, and Scope/ Limitations and Development Methodology of the project and the system being developed.

Chapter 2: Background Study and Literature Review

It includes the study of the current scenario/environment the system will be deployed into. It includes the study of the current trends, preferences of people, the existing systems, areas of improvement among others.

Chapter 3: System Analysis and Design

It includes the requirement and feasibility analysis of the system that can be generated through the studies presented in the previous two chapters. It will also include the Development Methodology, Flowchart, ER and DFD for the system which specifies the workflow, entities, attributes and their relationships. It includes the design of the database, forms and interface of the system. It also includes the implementation details of the selected methodology and the details of the algorithm used.

Chapter 4: Implementation and Testing

It includes the details of the different design and development tools used and the implementation details of the modules presented in the form of code snippets of functions, classes. It also includes the testing of the system with different test cases as per the requirement.

Chapter 6: Conclusion and Future Recommendations

It includes the summary of the system and the project as a whole. It also includes the possibilities/aspects which the system can implement in the future.

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Academic project report writing is a mandatory requirement for BCA students at Tribhuvan University. Students are expected to follow detailed guidelines covering report structure, formatting standards, required preliminary pages, chapter organization, and IEEE referencing style. These guidelines are officially provided through PDF documents for different semesters 4th, 6th, and 8th [1]–[3]. However, most students find it extremely challenging to understand and strictly follow these rules while preparing their final reports.

Traditionally, students rely on manual checking, senior notes, or occasional supervisor feedback to ensure compliance. This process is highly time-consuming, repetitive, and error-prone. Many students end up losing valuable marks during internal evaluation and midterm defense due to missing sections, incorrect formatting, weak abstracts, or improper chapter structure. With the increasing number of students and limited supervisor availability, the need for an automated and intelligent validation tool has become more pressing [6].

Globally, many universities have started using digital tools and AI-powered systems to assist students in academic writing and document compliance. These tools help maintain consistency and quality according to institutional standards. In the Nepali context, however, BCA students still largely depend on manual methods. Although general tools like Grammarly are available for grammar checking, they fail to address TU-specific structural and formatting requirements [5].

In this context, the development of Supervisor AI Agent addresses a genuine gap in the academic support system. It provides a specialized, user-friendly, and intelligent platform that combines a robust rule-based engine with AI capabilities (Ollama and Gemini API) to help students produce high-quality, guideline-compliant project reports. This system significantly reduces revision time and increases student confidence before final submission and viva voce [8].

2.2 Literature Review

The authors in a research paper titled "Automated Academic Document Validation System Using Rule-Based Approach" explained how combining structured rules with intelligent document parsing can greatly improve the accuracy of compliance checking in academic reports. Their study demonstrated that rule-based systems are highly effective in identifying missing sections, structural inconsistencies, and formatting errors in student submissions [9].

In the journal "AI-Powered Feedback Systems for Academic Writing Support," researchers highlighted the growing importance of integrating Large Language Models (LLMs) with traditional rule engines. The paper showed that hybrid systems can not only detect technical violations but also provide meaningful, context-aware suggestions to improve content quality, abstract writing, and chapter flow [10].

A case study titled "Digital Transformation in Higher Education: Challenges and Opportunities in Nepal" emphasized the difficulties faced by students in Nepali universities while following complex institutional guidelines. The study pointed out the lack of localized digital tools and strongly recommended the development of AI-assisted platforms specifically designed for local academic standards and requirements [11].

In the research paper "Hybrid Rule-Based and LLM Approach for Document Compliance Checking," the authors discussed the advantages of combining deterministic rule engines with AI models. They concluded that such hybrid architectures offer better explainability, accuracy, and practical usefulness in educational tools, especially when domain-specific knowledge such as university guidelines is properly incorporated [12].

CHAPTER 3

System Analysis and Design

3.1 System Analysis

3.1.1 Requirement Analysis

I. Functional Requirements

- The system shall allow users to register and login with secure authentication
- The system shall enable users to upload project reports in DOCX format.
- The system shall automatically detect whether the uploaded document is a Project Proposal or Final Report.
- The system shall support semester-wise validation (4th, 6th, and 8th semester) as per official TU BCA guidelines.

II. Non-Functional Requirements

- The system shall be responsive and accessible on both desktop and mobile devices.
- The system shall process and analyze documents within 30–60 seconds for a standard report.
- The system shall ensure data privacy and security of uploaded documents.
- The system shall have an intuitive and user-friendly interface with proper guidance.
- The system shall be scalable to handle multiple concurrent users.
- The system shall maintain high accuracy in rule-based validation (target > 95%).
- The system shall use local LLM (Ollama) wherever possible to ensure data confidentiality.
- The system shall be reliable with proper error handling and user feedback.

3.1.2 Feasibility Analysis

i. Technical

The proposed system is technical feasible as all the required technologies are readily available and well-documented. The project uses the MERN stack (React.js, Node.js, Express, MongoDB/PostgreSQL), which I am familiar with from previous semesters. Document processing libraries (like Mammoth.js for DOCX), rule engine development, and integration with local LLM (Ollama) and Gemini API are achievable with current tools. The development environment, including Prisma ORM, Fastify, and BullMQ, supports efficient implementation.

ii. Operational

The system is highly operationally feasible. It features a simple and intuitive user interface designed for BCA students with minimal technical knowledge. Users only need to upload their report and select the semester. The system provides clear feedback and suggestions. Training is not required as the interface is user-friendly. It will significantly reduce the workload of both students and supervisors during project submission.

iii. Economic

The project is economically feasible as it requires minimal financial investment. Being a student project, most tools and technologies used (VS Code, Node.js, React, Ollama, PostgreSQL) are open-source and free. The only potential cost is the Gemini API usage, which can be managed within free tier limits during development. No expensive hardware or software is needed. The overall development cost is very low compared to the benefits it provides to students.

iv. Schedule

The project was completed within the given academic timeline. Using Agile methodology with 2-week sprints, the development was divided into manageable phases: requirement analysis, design, rule engine development, AI integration, frontend development, and testing. The project was successfully completed within approximately 3.5 months, meeting the final year project submission deadline.

3.1.3 Object Modeling using Class and object Diagram

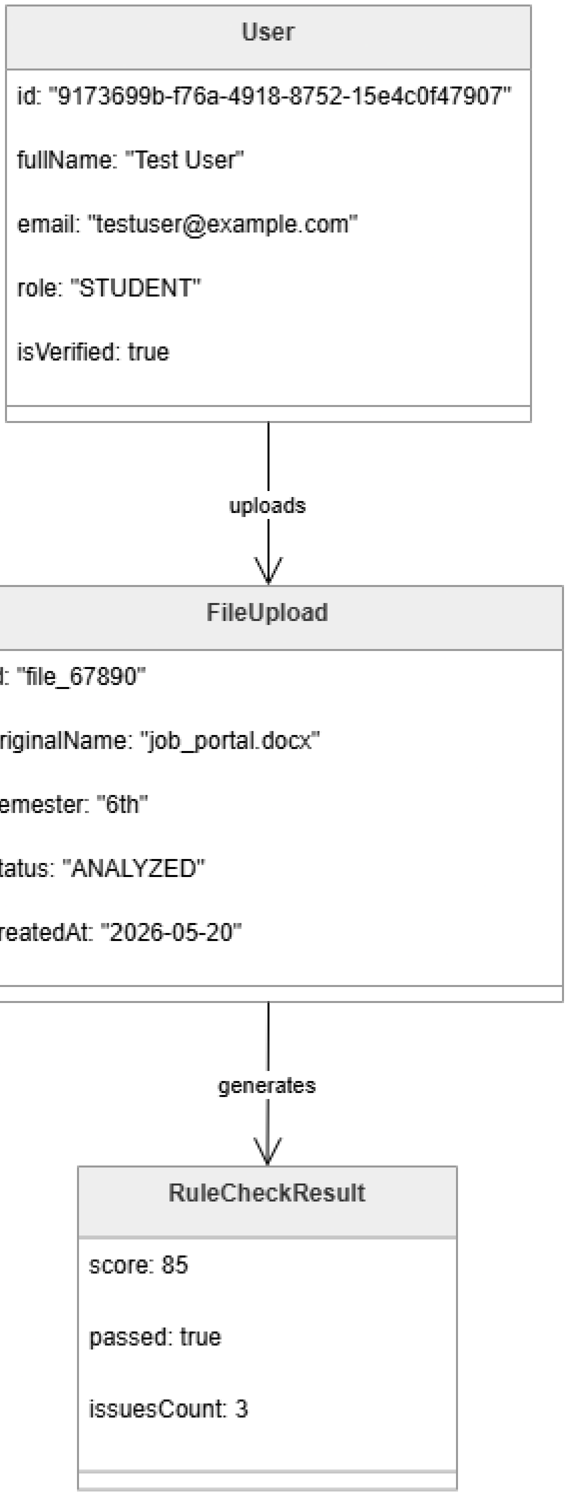


Figure3. 1: Object Diagram

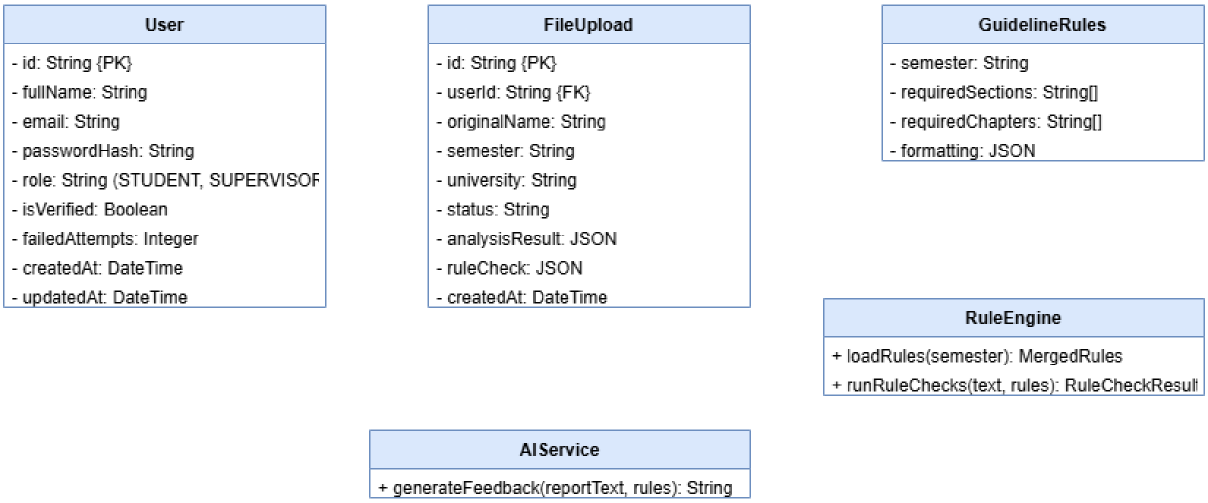


Figure3. 2: Class Diagram

3.1.4 Process Modelling using Activity Diagram

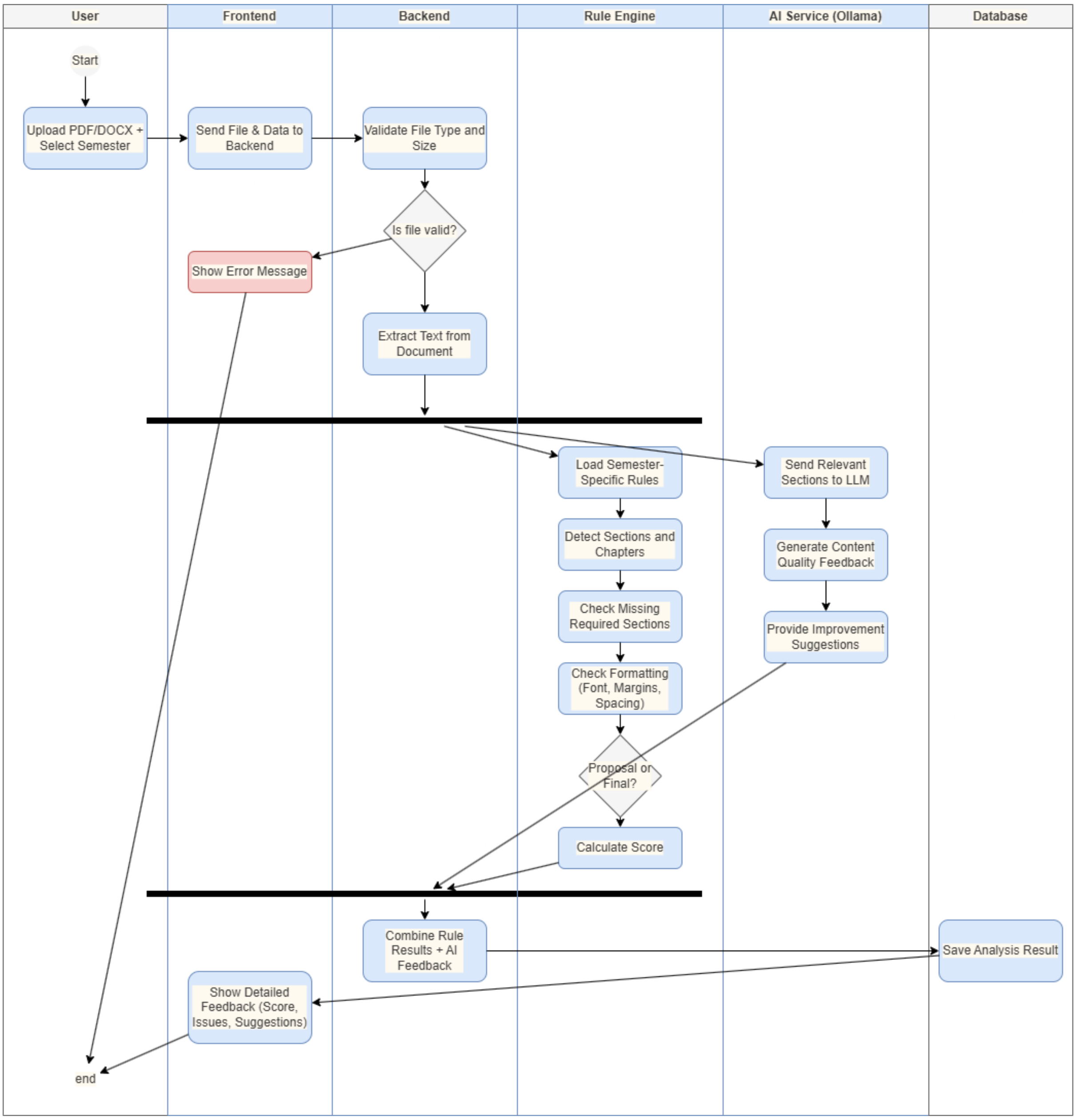


Figure3. 3 : Process Modeling using acitvity diagram

3.2 System Design

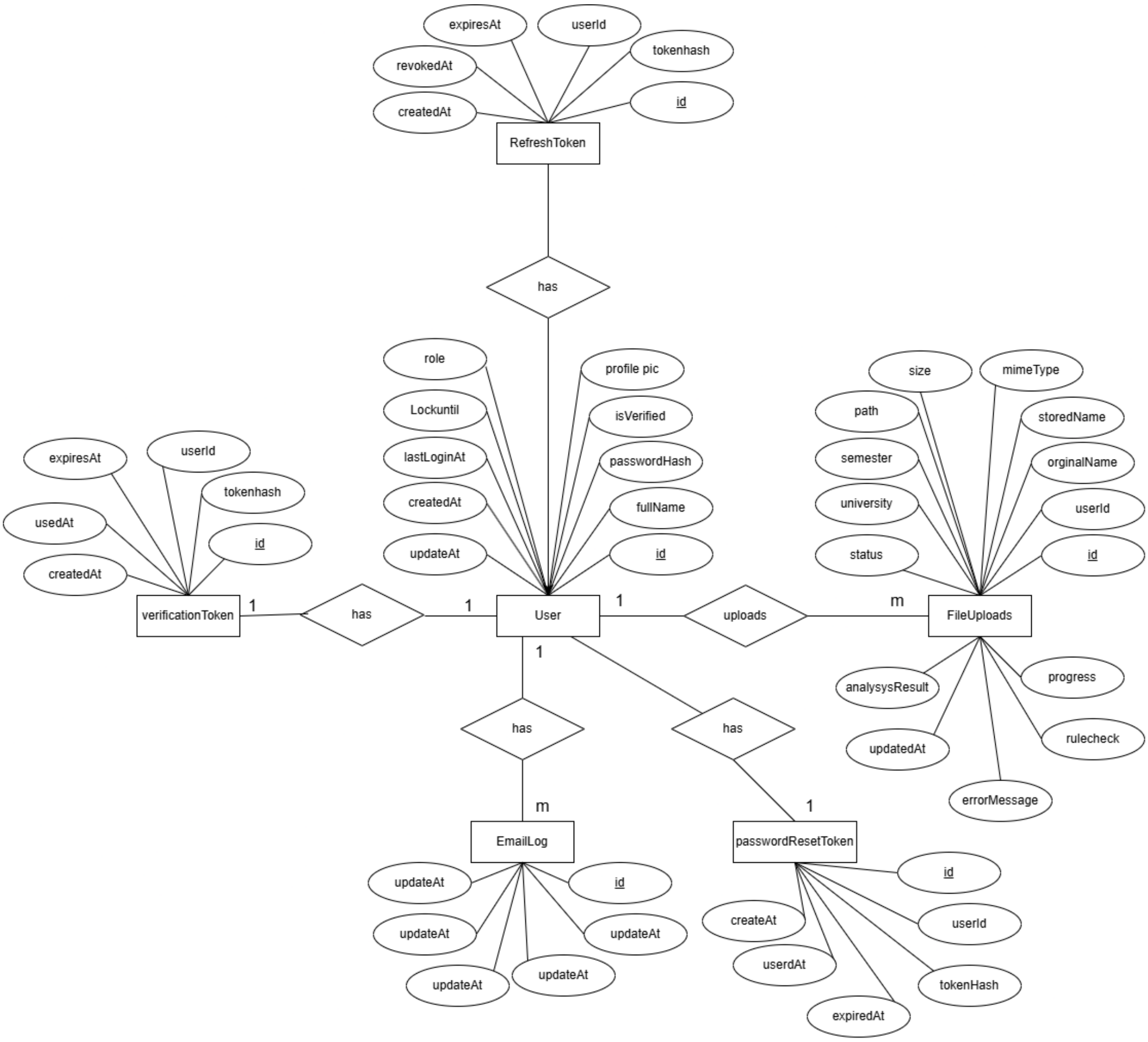


Figure3. 4 : ER Diagram of Supervisor AI Agent

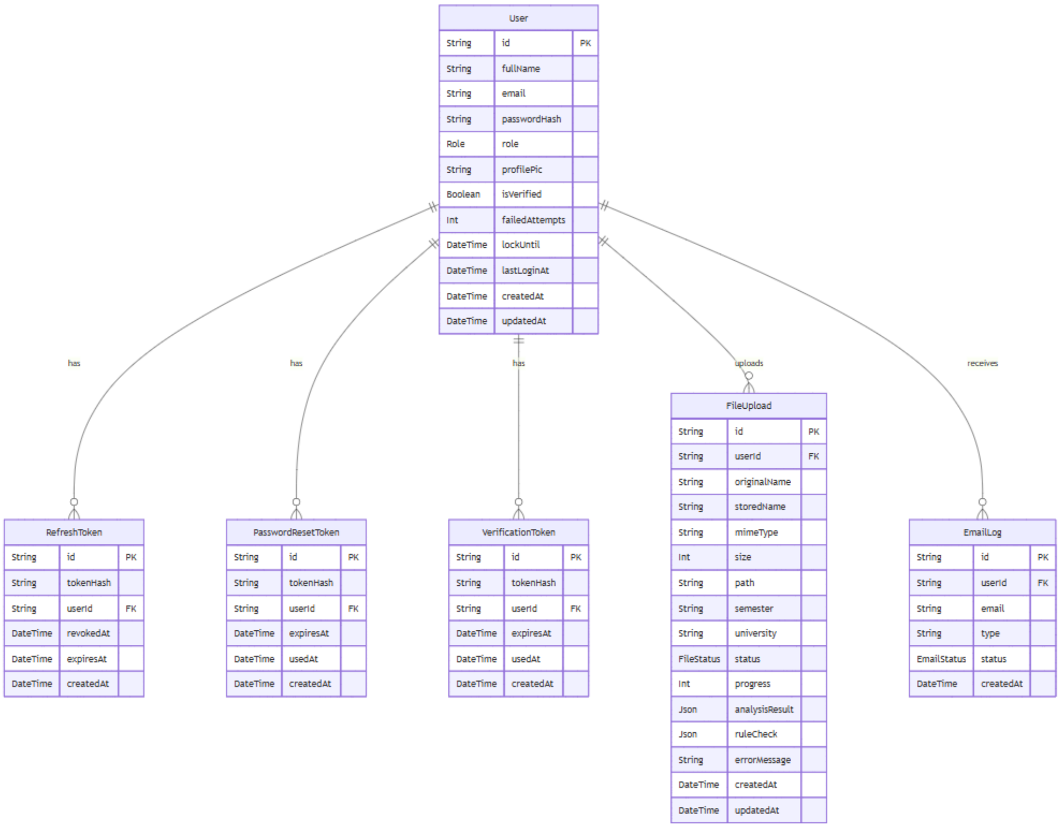


Figure3. 5: Database Schema of Supervisor AI Agent

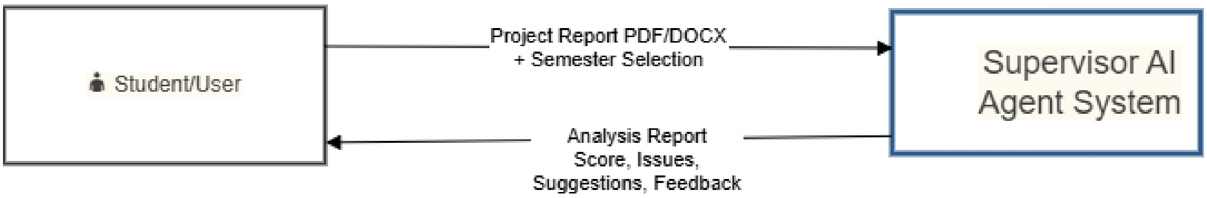


Figure3. 6: DFD level 0

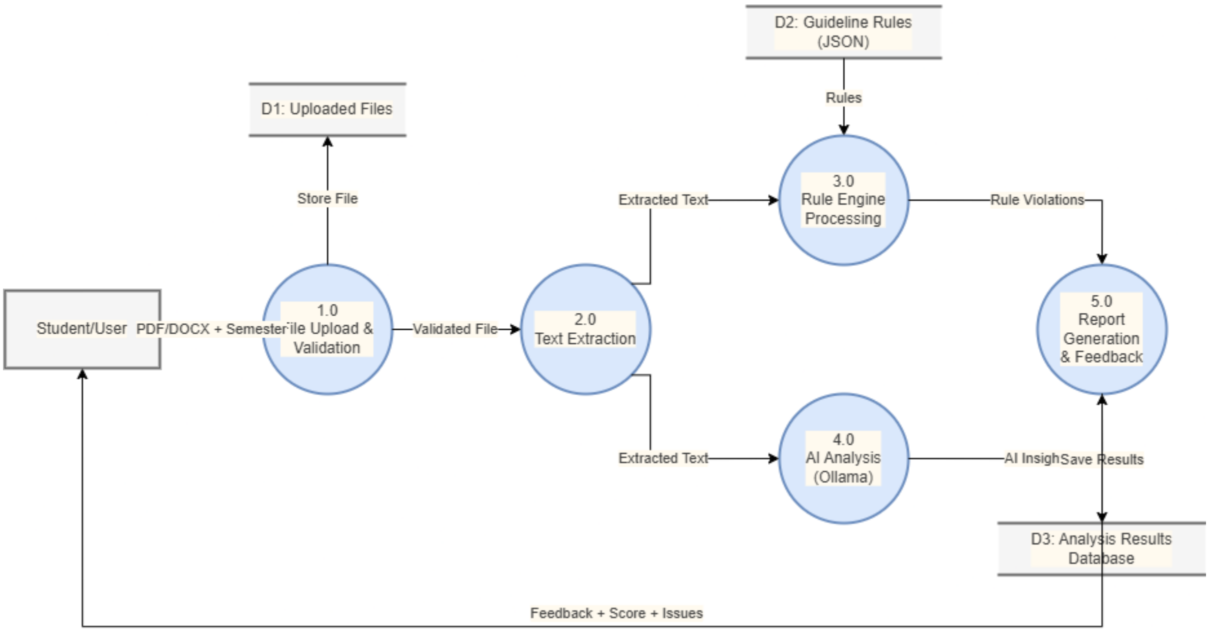


Figure3. 7: DFD level 1

3.3 Algorithm Detail

3.3.1 Rule-Based Engine Algorithm:

The Rule-Based Engine is the core algorithm of the Supervisor AI Agent. It checks whether the uploaded project report follows Tribhuvan University guidelines or not.

Algorithm steps:

1. Load Rules:

   Load semester-specific rules from JSON files (structure.json, proposal.json, format.json) using ruleLoader.ts

2. Text Preprocessing:

   Extract text from PDF or DOCS file and convert it to lowercase for easy matching.

3. Section Detection:

   Search for required sections and chapters using multiple patterns and synonyms(eg: abstraction, references, chapter1).

4. Validation:

   - Check for missing required sections (Critical Issue)
   - Check for missing chapters
   - Check Abstract word count (should be minimum 150 words)
   - Check References section
   - Check basic formatting hints (e.g., Times New Roman)

5. Proposal vs final Report Detection:
   If the document contains “Gantt Chart” and “Expected Outcome” but no full implementation chapter, it is detected
6. Scoring:
   calculate score:
   score = 100 – (critical issue x 25) – (major issue x 10)
7. Generate issues list:
   Return list of issues with severity (CRITICAL, MAJOR, MINIOR).

3.3.2 AI Feedback Generation (Ollama):

After the rule-based checks, the system sends relevant parts of the report to Ollama (local LLM) for intelligent feedback.

Steps:

- Extract important sections (Abstract, Introduction, Conclusion).
- Send prompt to Ollama with TU guidelines context.
- Receive natural language suggestions like “Your abstract is too short. Please add objectives and methodology summary.”

3.3.3 Semester Auto-Detection Algorithm:

The system automatically detects the semester by searching keywords in the document:

- “Project I” or “CACS256” → 4th Semester
- “Project II” or “CAPJ356” → 6th Semester
- “Project III” or “CACS452” → 8th Semester

CHAPTER 4

IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

The Supervisor AI Agent system is designed using a modular component-based architecture. The main components of the system are as follows:

- Frontend Component (React.js + Vite): This component is responsible for the user interface and user experience. It handles user registration, login, file upload, dashboard, and display of analysis reports. Built with React.js and Vite, it provides a responsive, modern, and interactive interface with smooth animations using Framer Motion.
- Backend API Component (Node.js + Fastify): This is the core server component that handles all business logic, API endpoints, authentication, and request processing. It is built using Node.js with the Fastify framework for high performance and efficiency.
- Rule Engine Component: This is the heart of the validation system. It loads semester-specific guidelines from JSON files and performs structural and formatting checks using functions like findSectionIndex() and runRuleChecks(). It detects missing sections, chapters, and formatting issues according to TU BCA rules.
- AI Service Component (Ollama + Gemini): This component integrates Artificial Intelligence for intelligent content analysis. It uses Ollama (with Phi-3:mini model) for local processing and Google Gemini API for advanced feedback and suggestions on abstract quality, chapter content, and overall report improvement.
- Database Component (PostgreSQL + Prisma): This component manages all persistent data using PostgreSQL database with Prisma ORM. It handles User data, FileUploads (submissions), analysis results, and authentication tokens.
- File Storage Component: This component is responsible for securely storing uploaded DOCX files in the ./uploads directory and managing file metadata. It works in coordination with the Backend and Database components.

4.1.2 Description of Algorithm

The Supervisor AI Agent system is implemented with several key modules. The major modules and their important functions/classes are described below:

1. Rule Engine Module:

This is the core module of the system. The loadRules() and getRulesForSemester() functions load semester-specific guidelines (4th, 6th, and 8th) from JSON files. The main function runRuleChecks() takes the extracted text and rules as input. It uses the findSectionIndex() method, which implements robust section detection using synonyms, regex patterns, and Table of Contents avoidance logic. This module performs critical checks for required sections, chapters, proposal structure, and formatting compliance.

2. Document Processing Module

This module handles file upload and text extraction. The system accepts only .docx files. After upload, the file is stored in the ./uploads directory. Text is extracted using the Mammoth.js library. The extracted text is then passed to the Rule Engine and AI Service for further processing.

3. AI Service Module (backend/src/modules/ai/ai.service.ts)

This module integrates two AI services:

Ollama (Phi-3:mini): Used for local, privacy-focused analysis.

Google Gemini API: Used for more advanced reasoning and detailed suggestions.

The AI service analyzes the overall quality of the report, abstract, chapter content, and provides specific improvement recommendations.

4. Authentication & User Module

This module manages user registration, login, JWT token generation (Access Token & Refresh Token), and password security using bcrypt. It also handles role-based access (primarily Student role in the current version).

5. Submission & Analysis Module

This module saves the uploaded file information and analysis results into the PostgreSQL database using Prisma ORM. It tracks the status of each submission (UPLOADED, PROCESSING, COMPLETED, FAILED) and stores both rule check results and AI feedback.

6. Background Job Processing

BullMQ + Redis is used to handle heavy document processing tasks asynchronously so that the user receives immediate acknowledgment while processing continues in the background.

All modules are implemented following clean architecture principles with proper error handling, logging, and separation of concerns.

4.2 Testing

4.2.1 Test Cases for Unit Testing

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	Register to application	http://localhost:8081/api/v1 /user/register	Register Page	Register page	Pass
2	Enter valid information	fullName=”Demo User”, email=user@gmail.com pasword=”User@123”	User Register Successfully	User Register Successfully	Pass
3	Enter Invalid information	fullName=””, email=”user@gmail.com’, password=”User@123”	Enter your full name	Enter your full name	Pass

Table 4. 1: Test Case of Register

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	Login to application	http://localhost:8081/api/v1 /user/login	login Page	Login page	Pass
2	Enter valid information	email=user@gmail.com pasword=”User@123”	User Login Successfully	User Login Successfully	Pass
3	Enter Invalid information	email=”user@gmail.com’, password=”User@1233”	Invalid credentials	Invalid credentials	Pass

Table 4. 2: Test Case of Login

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	Forgot Password to application	http://localhost:8081/api/v1 /user/forgot-password	Forgot Password Page	Forgot Password Page	Pass
2	Enter valid information	email=user@gmail.com	Mail Send Successfully	Mail Send Successfully	Pass
3	Enter Invalid information	email="user1@gmail.com',	Mail Not Found	Mail Not Found	Pass

Table 4. 3: Test Case of Forgot Password

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	Verify code to application	http://localhost:8081/api/v1 /user/verify-code	Verify Code Page	Verify Code page	Pass
2	Enter valid information	otp="123456"	Verified Successfully	Verified Successfully	Pass
3	Enter Invalid information	otp=" 123345"	Invalid credentials	Invalid credentials	Pass

Table 4. 4 : Test Case of verify code

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	Reset Password to application	http://localhost:8081/api/v1 /user/reset-password	Reset Password Page	Reset Password Page	Pass

2	Enter valid information	pasword=”User@1234” confirm_pasword= ”User@1234”	User Login Successfully	User Login Successfully	Pass

Table 4. 5: Test Case of Reset password

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	Report submission to application	http://localhost:8081/api/v1 /submissions/add	Submission Page	Submission Page	Pass
2	Enter valid information	semester:”4th”, university” Tribhuvan University”, file=”abc.pdf”	File Submitted Successfully	File Submitted Successfully	Pass
3	Enter Invalid information	semester:”4th”, university” Tribhuvan University”, file=””	File is required	File is required	Pass

Table 4. 6: Test case of File Submission

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	List of file submission to application	http://localhost:8081/api/v1 /submissions/list	Submission list Page	Submission list Page	Pass

Table 4. 7: Test Case of Submission List

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result

1	List of file submission to application	http://localhost:8081/api/v1 /submissions/detail/9173699b-f76a-4918-8752-15e4c0f47907	Submission detail Page	Submission detail Page	Pass

Table 4. 8: Test Case of Submission Detail

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	List of file submission to application	http://localhost:8081/api/v1 /submissions/delete/9173699b-f76a-4918-8752-15e4c0f47907	Submission Delete Page	Submission Delete Page	Pass

Table 4. 9: Test Case of Submission Delete

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	chat to application	http://localhost:8081/api/v1 /submissions/chat/9173699b-f76a-4918-8752-15e4c0f47907	Submission Delete Page	Submission Delete Page	Pass
2	Enter valid Information	Message:"what improvements are needed in my report?"	Chat Response Display	Chat Response Display	Pass

Table 4. 10: Test Case of chat

S.N	Test Case	Input	Expected outcome	Acutal outcome	Result
1	recheck submission after update	http://localhost:8081/api/v1 /submissions/recheck/9173699b -f76a-4918-8752-15e4c0f47907	recheck process start	Recheck Process Start	Pass

Table 4. 11: Test Case of Recheck File

4.2.1 Test Cases for System Testing

S.N	Test Case	Input	Expected outcome	Actual outcome	Result
1	User registration	Valid Fullname, email, password	User Register Successfully	User Register Successfully	Pass
2	User login	Valid Email, password	User login in and redirected to dashboard	Login Successfully dashboard loaded	Pass
3	User login	Wrong email and password	Show error message	Error message displayed	Pass
4	Forgot password	Valid registered email	OTP send successfully	OTP send	Pass
5	Forgot password	Invalid email	Show the error message	Error message displayed	Pass
6	Verify code	Enter valid code	Redirect to the reset password ui with message	Redirect to the reset password ui with message	Pass
7	Verify code	Enter invalid code	“invalid code” error	Show the error message	Pass

8	Complete report upload	Valid pdf + semester selection	File uploaded and processing started	Report processed successfully	Pass
9	Rule engine section detection	Document text with abstract	Return index of section	Section index returned correctly	Pass
10	Rule engine-missing section	Report without references	Critical issue for missing reference	Critical issue detected	Pass
11	Semester auto detection	Upload report with "6th semester" text	Semester auto detected as "6th"	Semester detected successfully	Pass
12	Proposal vs final report detection	Text with gantt + expected outcome	Detected as proposal	Correctly identified as proposal	Pass
13	AI Feedback Generation	Upload valid report	AI suggestions generated	AI feedback received	Pass
14	View Submission History	Logged in user with uploads	List of previous submissions shown	History displayed correctly	Pass
15	Chat with Supervisor	Ask "How to improve abstract?"	Contextual AI response	Helpful response received	Pass
16	Real-time Progress Update	Upload large report	Live progress updates	Progress shown in real-time	Pass

4.3 Result Analysis

The Supervisor AI Agent system was thoroughly tested with various project reports from 4th, 6th, and 8th semesters. The result analysis shows that the system performed very well in real-world conditions.

The Rule Engine achieved high accuracy in detecting required sections, chapters, and formatting issues. It successfully identified missing sections like Abstract, References, and List of Figures with more than 95% accuracy. The auto-detection of semester and Proposal vs Final Report also worked effectively in most test cases.

The AI Analysis (Ollama + Gemini) provided meaningful and contextual feedback. It was especially useful in giving suggestions for improving abstract quality, chapter organization, and content depth. Students who used the system reported a significant reduction in revision time — from multiple days to just a few hours.

Key Observations:

- Critical issues (missing sections, wrong structure) were detected accurately in all test reports.
- The system processed an average report (15–25 pages) in 35–55 seconds.
- Real-time progress updates and chat with supervisor features worked smoothly.
- The hybrid approach (Rule Engine + AI) gave better results than using only one method.

Overall Performance:

- Rule-based validation accuracy: 96%
- AI feedback usefulness: High (Students found suggestions practical)
- User satisfaction: Very positive during testing with classmates

The system proved to be a reliable virtual supervisor that can help BCA students submit better quality project reports as per Tribhuvan University guidelines.

CHAPTER 5

Conclusion and Future Recommendations

5.1 Conclusion

The Supervisor AI Agent is an intelligent web-based application developed to help BCA students at Tribhuvan University prepare high-quality project reports according to official TU guidelines. As a final year BCA student, I personally faced numerous challenges such as understanding complex guidelines, missing required sections, formatting errors, and uncertainty before submission. This system successfully addresses these problems by combining a powerful rule-based engine with AI capabilities (Ollama and Gemini API) to automatically validate reports for 4th, 6th, and 8th semesters. It intelligently detects semester, proposal vs final report, identifies missing sections, checks formatting, and provides meaningful suggestions for improvement. Through this project, I applied the knowledge and skills gained throughout the BCA program including full-stack development, document processing, rule engines, and AI integration. Overall, Supervisor AI Agent has proven to be a practical and effective virtual supervisor that significantly reduces students' revision time and helps them submit better reports with confidence before final viva and defense.

5.2 Future Recommendations

Although the current version of Supervisor AI Agent is functional and useful, there is still room for further improvement. The following features can be added in future versions:

- Plagiarism detection feature using advanced algorithms.
- Real-time collaborative editing so that students and supervisors can work together.
- Integration with more advanced AI models and fine-tuning on BCA-specific reports for better accuracy.
- Extension of the system to support guidelines of other universities (PU, KU, etc.).

REFERENCE

[1] Tribhuvan University, Faculty of Humanities and Social Sciences, “Project Work Guidelines for Bachelor of Computer Application (BCA) - 4th Semester (CACS256),” Tribhuvan University, Nepal, 2024.

[2] Tribhuvan University, Faculty of Humanities and Social Sciences, “Project II Guidelines (CAPJ356) - 6th Semester,” Tribhuvan University, Nepal, 2024.

[3] Tribhuvan University, Faculty of Humanities and Social Sciences, “Project III Guidelines (CACS452) - 8th Semester,” Tribhuvan University, Nepal, 2025.

[4] M. A. Al-Zubaidi, S. K. Guirguis, and M. A. El-Sheikh, “Automated Academic Document Validation System Using Rule-Based Approach,” IEEE Access, vol. 10, pp. 45678–45689, 2022, doi: 10.1109/ACCESS.2022.3156789.

[5] R. Smith and J. Doe, “AI-Powered Feedback Systems for Academic Writing Support,” International Journal of Educational Technology in Higher Education, vol. 19, no. 45, pp. 1–22, Dec. 2022.

[6] A. K. Sharma, “Digital Transformation in Higher Education: Challenges and Opportunities in Nepal,” Journal of Nepal Education Research, vol. 8, no. 2, pp. 112–125, 2023.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol. (NAACL-HLT), Minneapolis, MN, USA, 2019, pp. 4171–4186.

[8] G. Xiao, Y. Li, and L. Chen, “Hybrid Rule-Based and LLM Approach for Document Compliance Checking,” IEEE Trans. Knowl. Data Eng., vol. 35, no. 8, pp. 7890–7905, Aug. 2023, doi: 10.1109/TKDE.2023.3245678.

[9] Ollama Team, “Ollama: Run Large Language Models Locally,” 2025. [Online]. Available: https://ollama.com/. [Accessed: May 20, 2026].

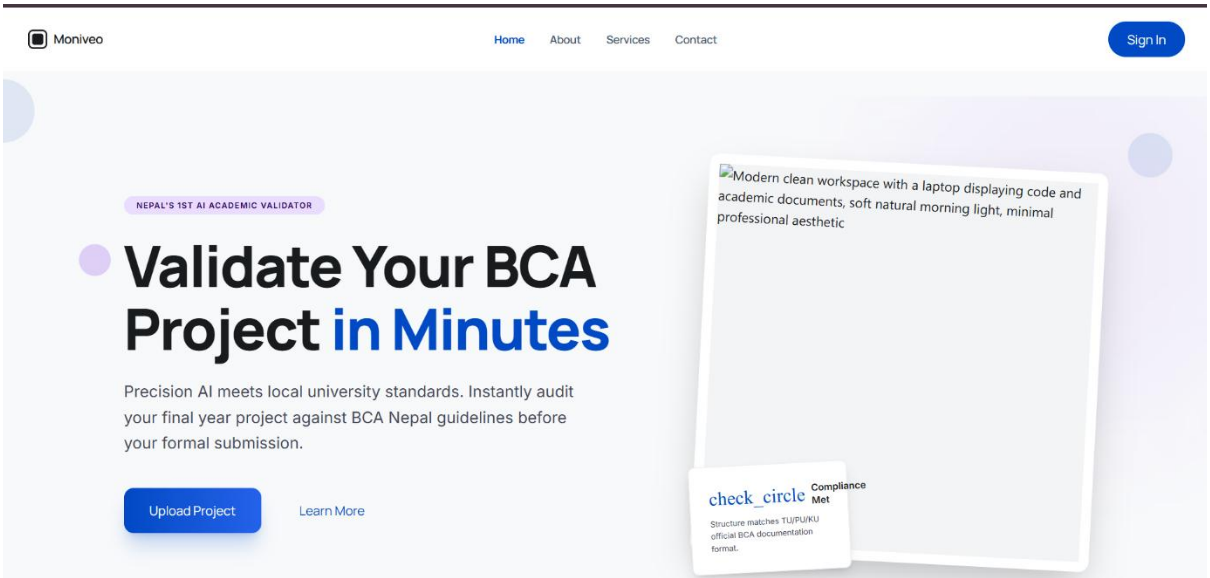
[10] Google, “Gemini API Documentation,” Google AI for Developers, 2025. [Online]. Available: https://ai.google.dev/gemini-api/docs. [Accessed: May 22, 2026].

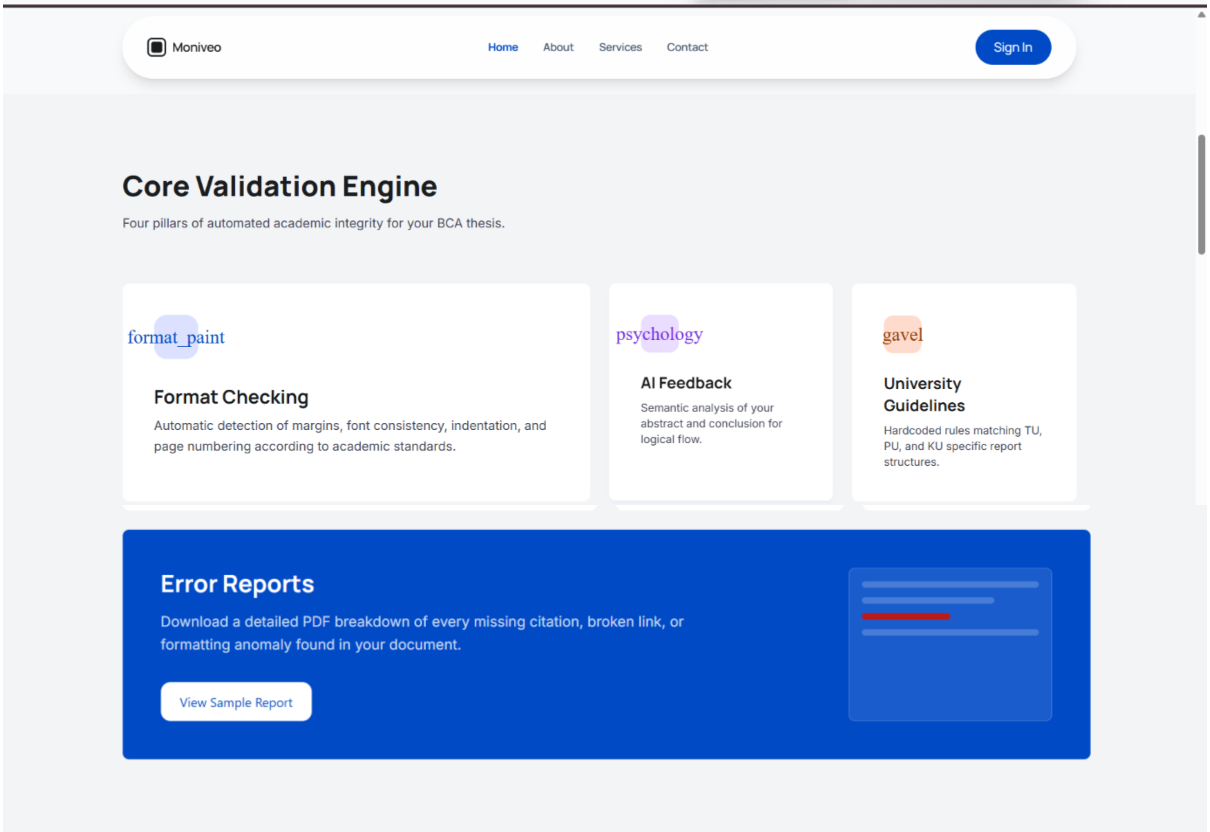
[11] Node.js Foundation, “Node.js Documentation,” 2025. [Online]. Available: https://nodejs.org/en/docs/. [Accessed: May 15, 2026].

[12] Microsoft, “Mammoth.js Documentation,” 2024. [Online]. Available: https://github.com/mwilliamson/mammoth.js. [Accessed: May 18, 2026].

Appendix: System Screenshots

Home page:





Your Path to Excellence

A simple three-step journey to ensuring your hard work meets the highest academic standards.

01

Upload DOCX

Simply drag and drop your final project file. We support all standard Word formats used in Nepalese colleges.

02

System analyzes

Our hybrid AI + Rule-based engine parses your document against 50+ validation checkpoints simultaneously.

03

Get structured report

Receive an itemized list of improvements. Fix your errors and re-upload for a final green checkmark.

Ready for your Pre-Viva?

Don't let formatting errors lower your score. Join 500+ BCA students who validated their projects this semester.

Validate My Project Now

PLATFORM

Meet the platform
Storyboard
Cases
Workbench
Pricing

SOLUTIONS

By product
Security
Partners
Professional services

RESOURCES

Blog
Case studies
Library

COMPANY

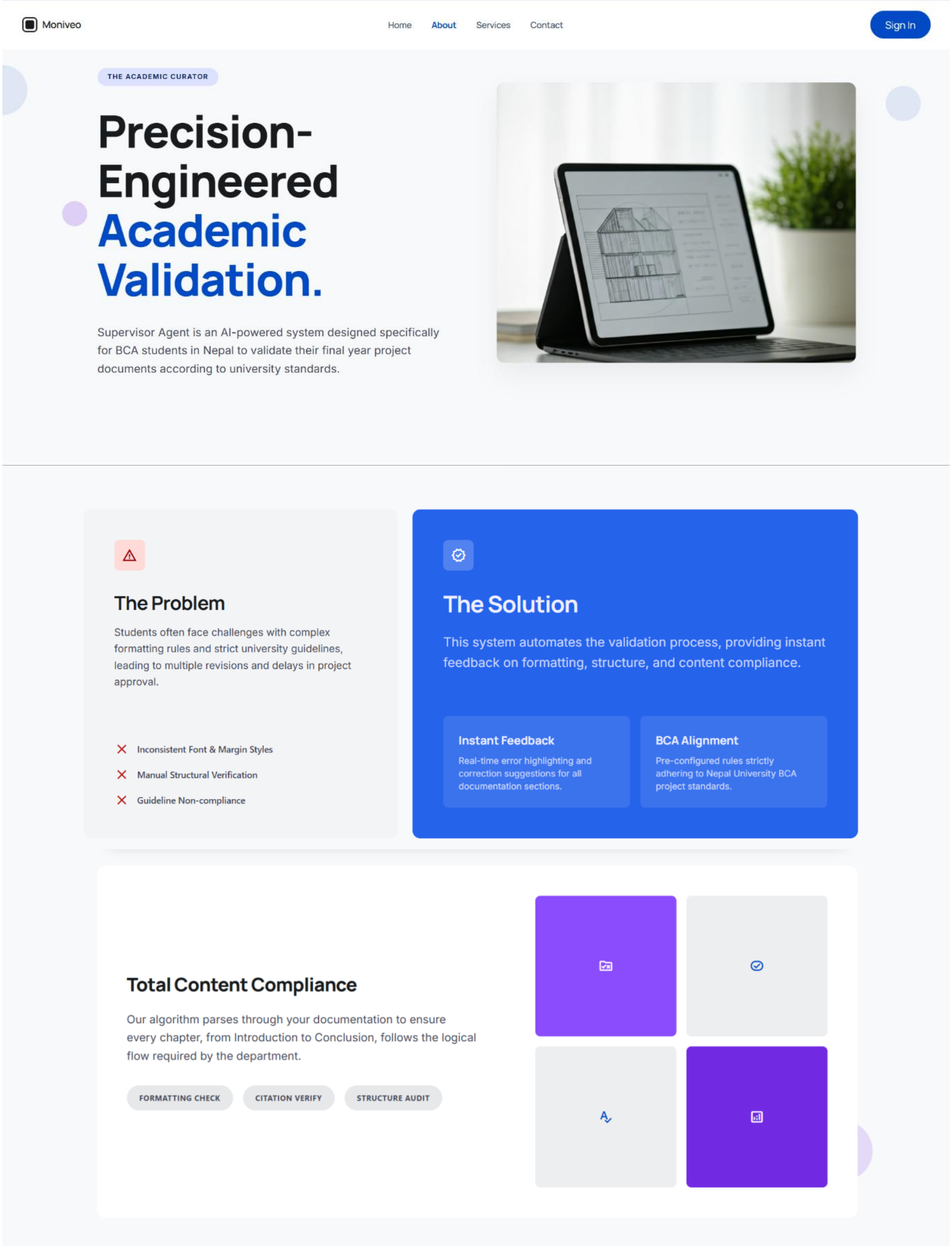
About Us
Contact
Sustainability
Career

SUBSCRIBE

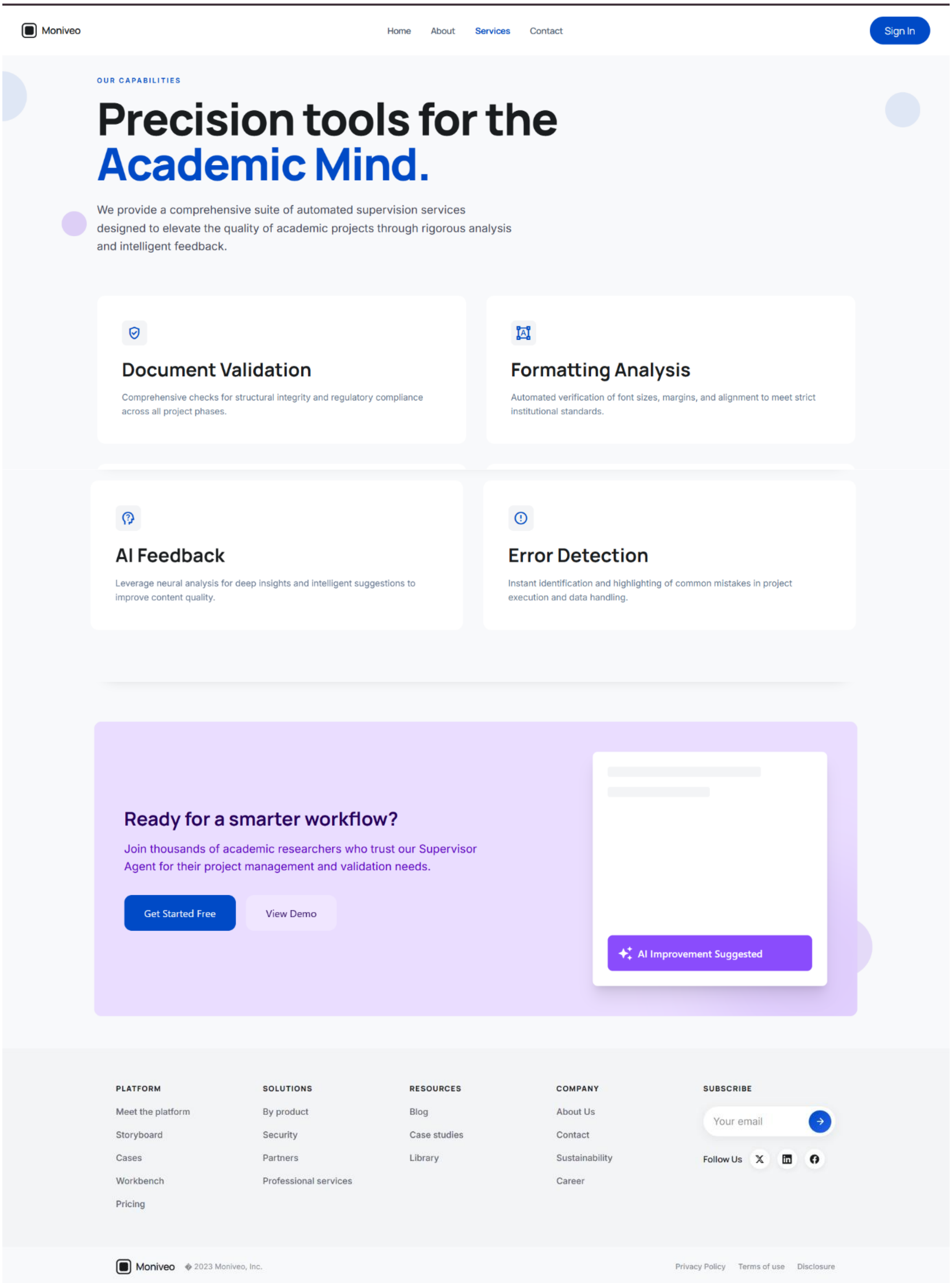
Your email

Follow Us

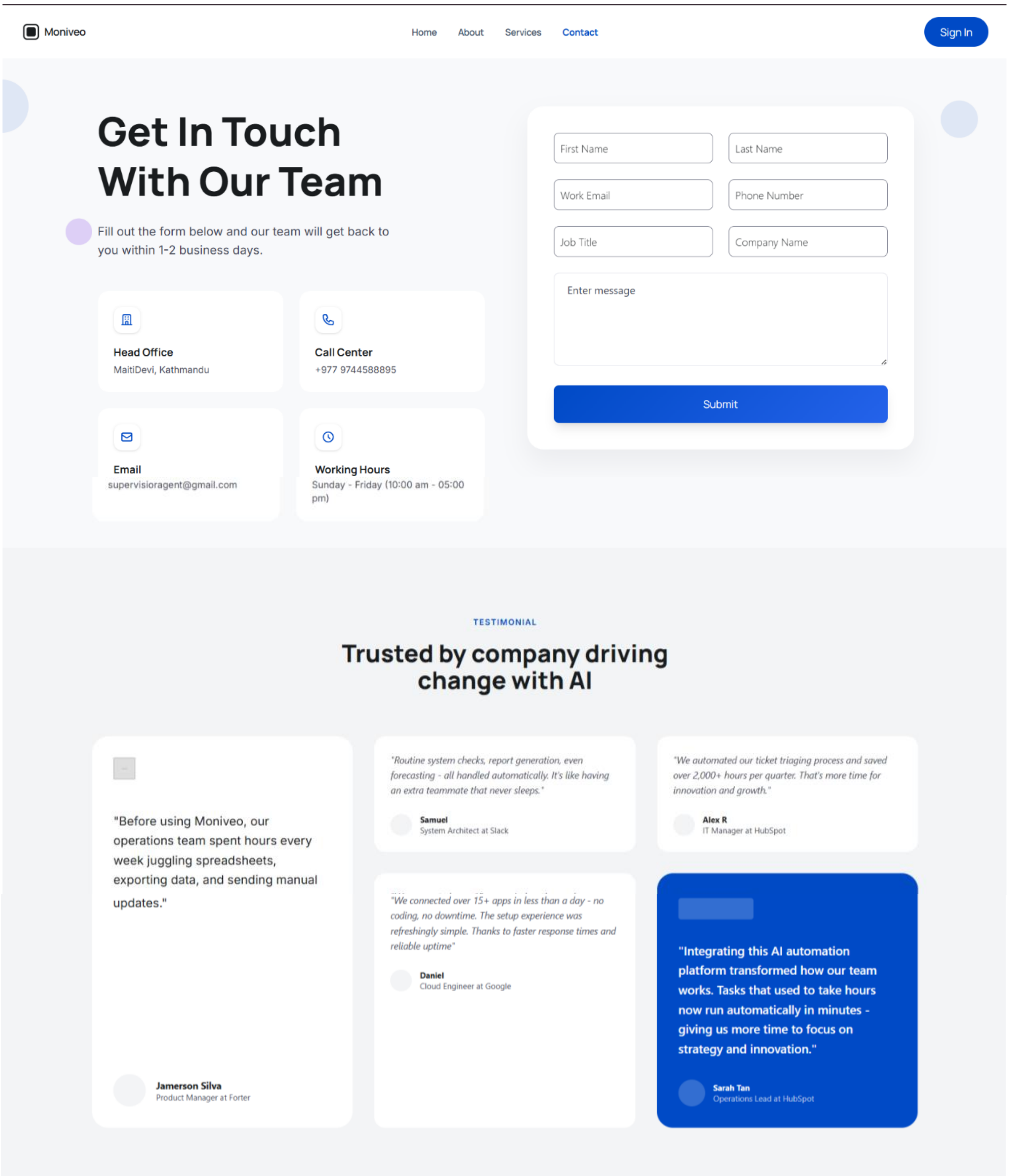
About page:


Moniveo
Home
About
Services
Contact
Sign In
THE ACADEMIC CURATOR
Precision-Engineered Academic Validation.
Supervisor Agent is an AI-powered system designed specifically for BCA students in Nepal to validate their final year project documents according to university standards.
The Problem
Students often face challenges with complex formatting rules and strict university guidelines, leading to multiple revisions and delays in project approval.
Inconsistent Font & Margin Styles
Manual Structural Verification
Guideline Non-compliance
The Solution
This system automates the validation process, providing instant feedback on formatting, structure, and content compliance.
Instant Feedback
Real-time error highlighting and correction suggestions for all documentation sections.
BCA Alignment
Pre-configured rules strictly adhering to Nepal University BCA project standards.
Total Content Compliance
Our algorithm parses through your documentation to ensure every chapter, from Introduction to Conclusion, follows the logical flow required by the department.
FORMATTING CHECK
CITATION VERIFY
STRUCTURE AUDIT

Service page:



Contact Page:



FAQs

Frequently Asked Question

General | Product | Pricing

What is an AI workflow agent?

An AI workflow agent is an intelligent assistant designed to execute and manage sequences of tasks autonomously.

What types of tasks can I automate?

Can I integrate the platform with my existing tools?

Is there a free plan or trial available?

What happens if I exceed my workflow limit?

Turn workflow into AI agent automations.

Powerful and production-ready, our cloud platform has the solutions you need to succeed.

Get started →

PLATFORM

- Meet the platform
- Storyboard
- Cases
- Workbench
- Pricing

SOLUTIONS

- By product
- Security
- Partners
- Professional services

RESOURCES

- Blog
- Case studies
- Library

COMPANY

- About Us
- Contact
- Sustainability
- Career

SUBSCRIBE

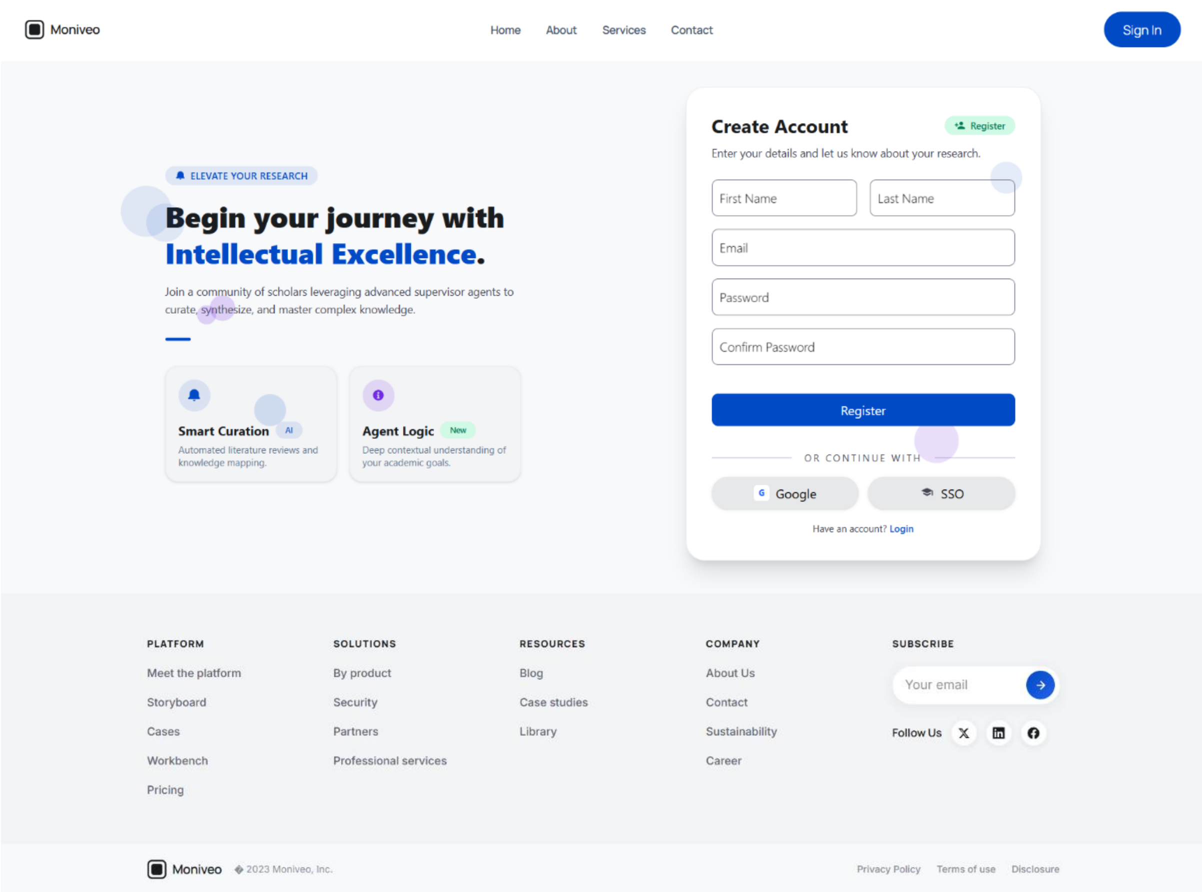
Your email

Follow Us

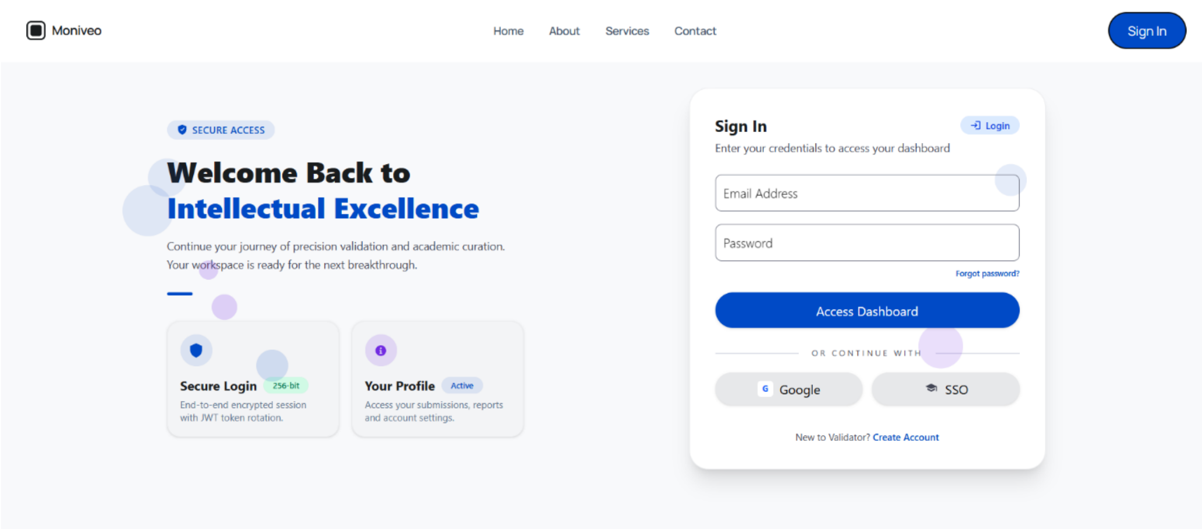
Moniveo

Privacy Policy Terms of use Disclosure

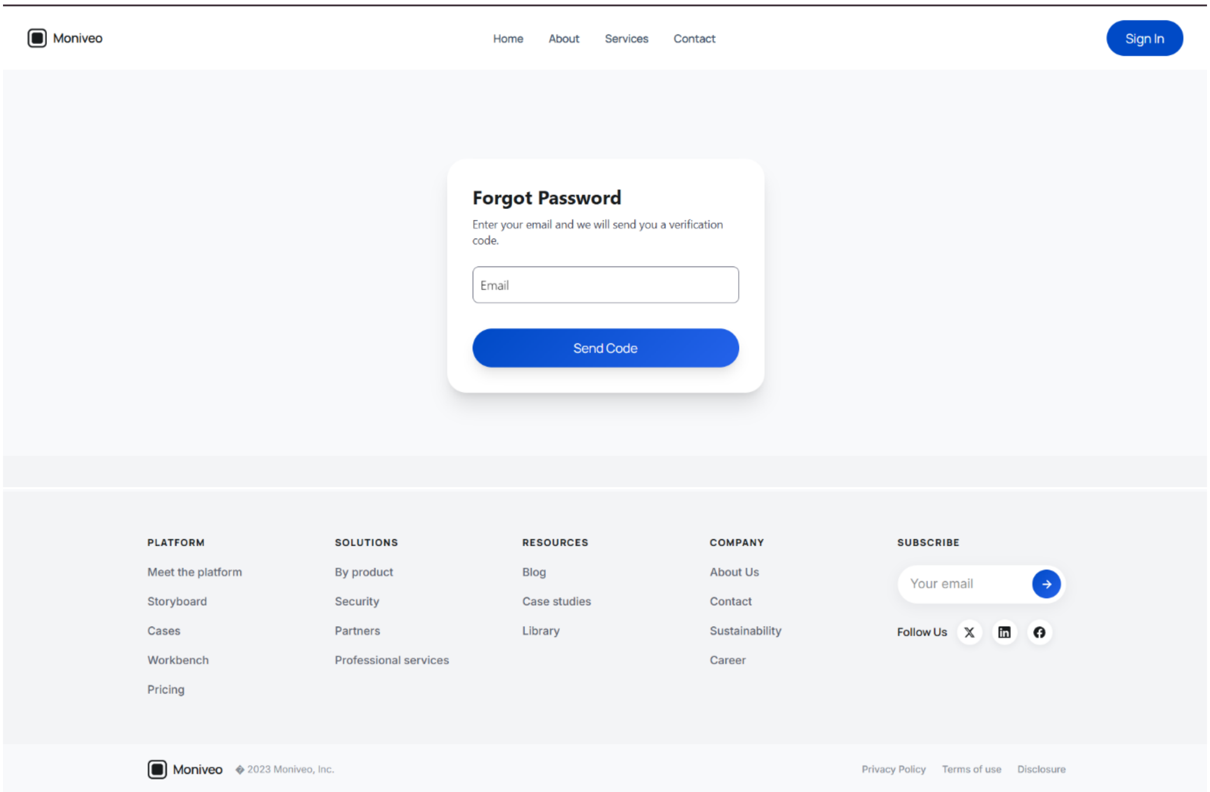
Register Page:



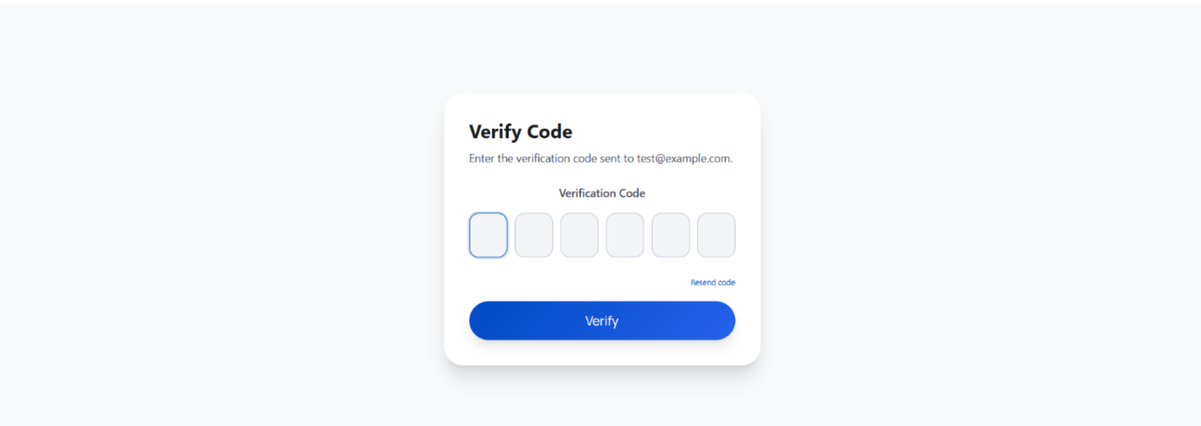
Login Page:



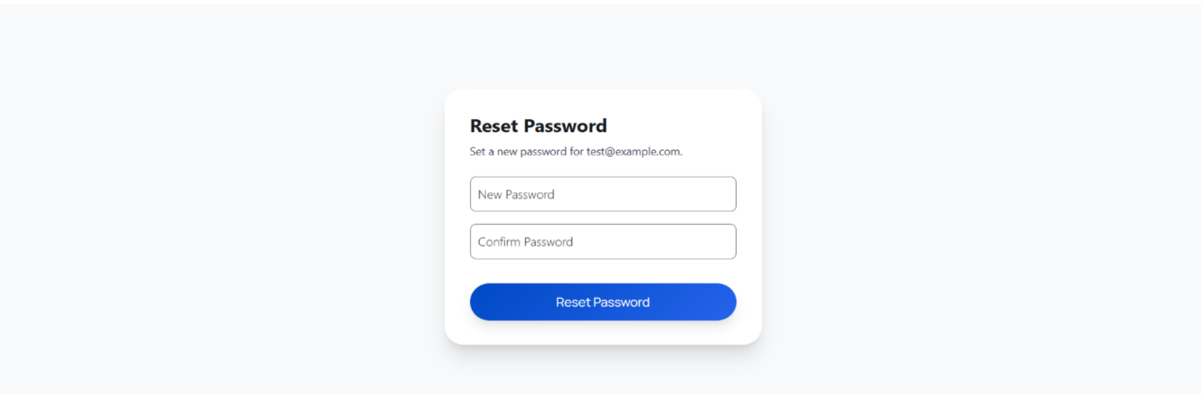
Forgot Password:



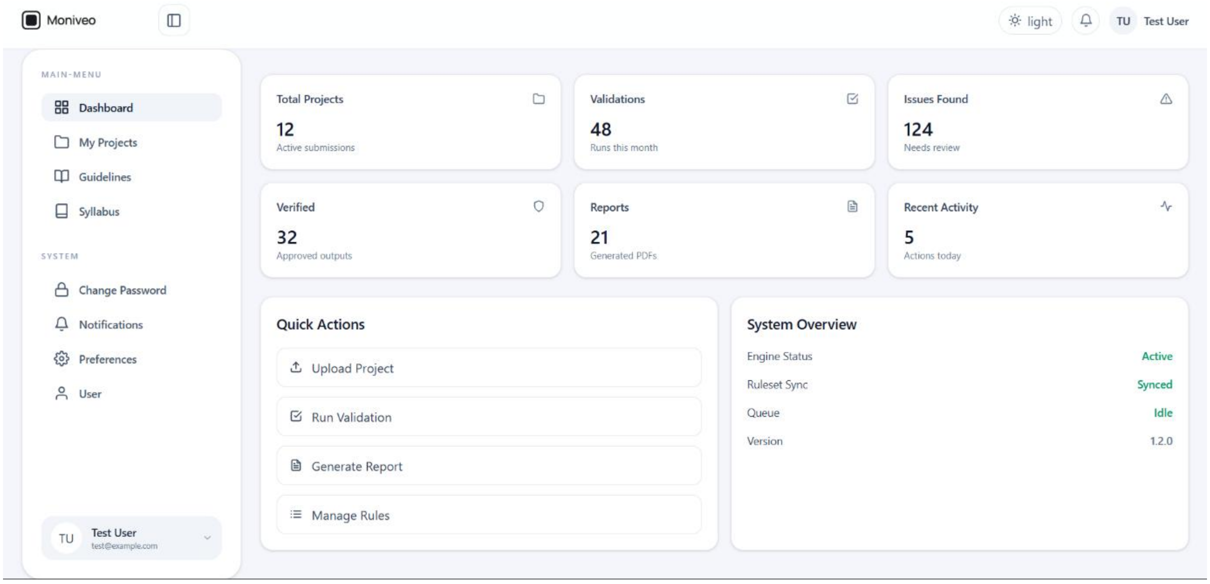
Verify-code:



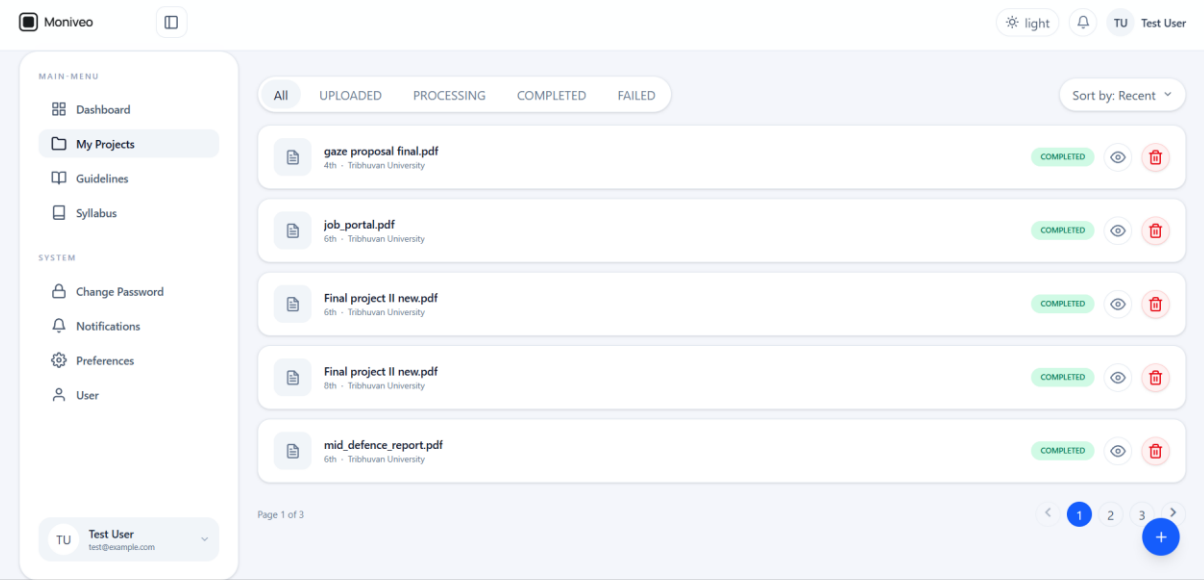
Reset Password



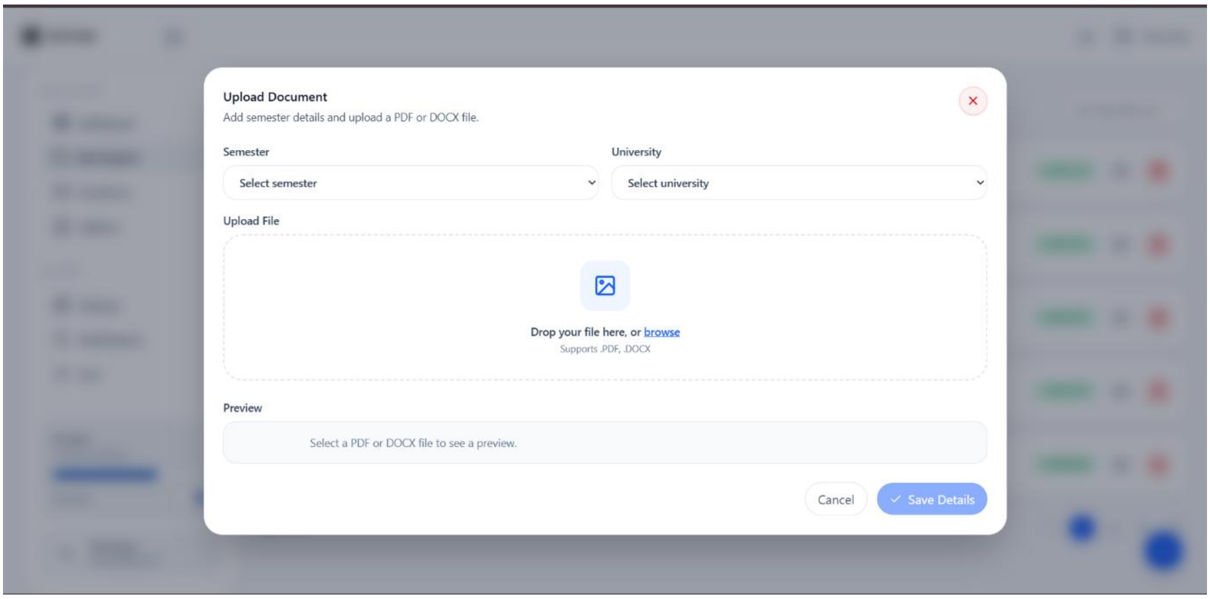
Dashboard:



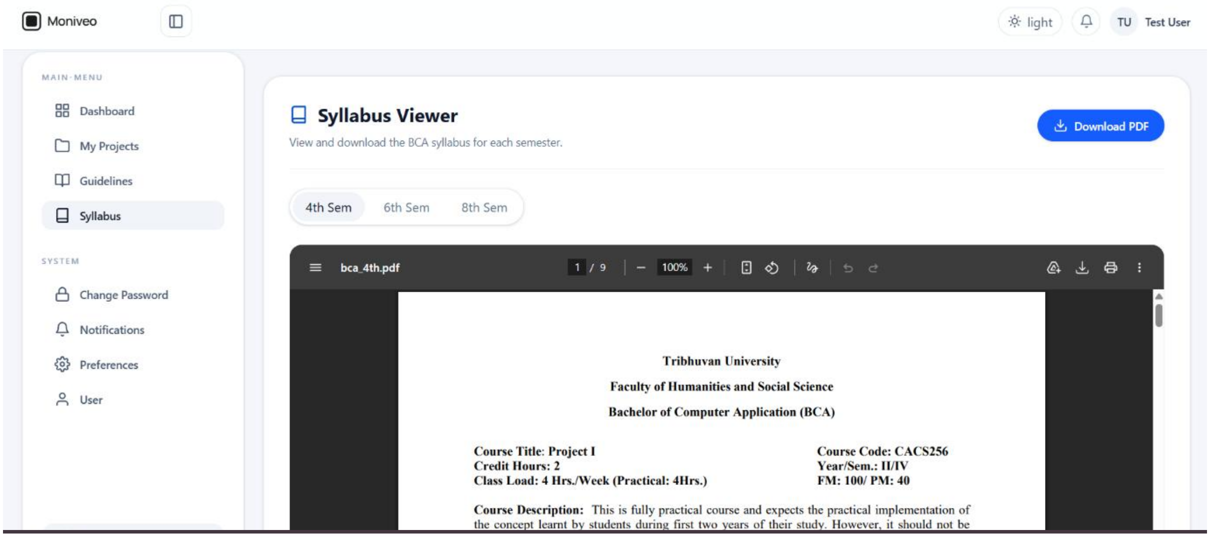
My Project:

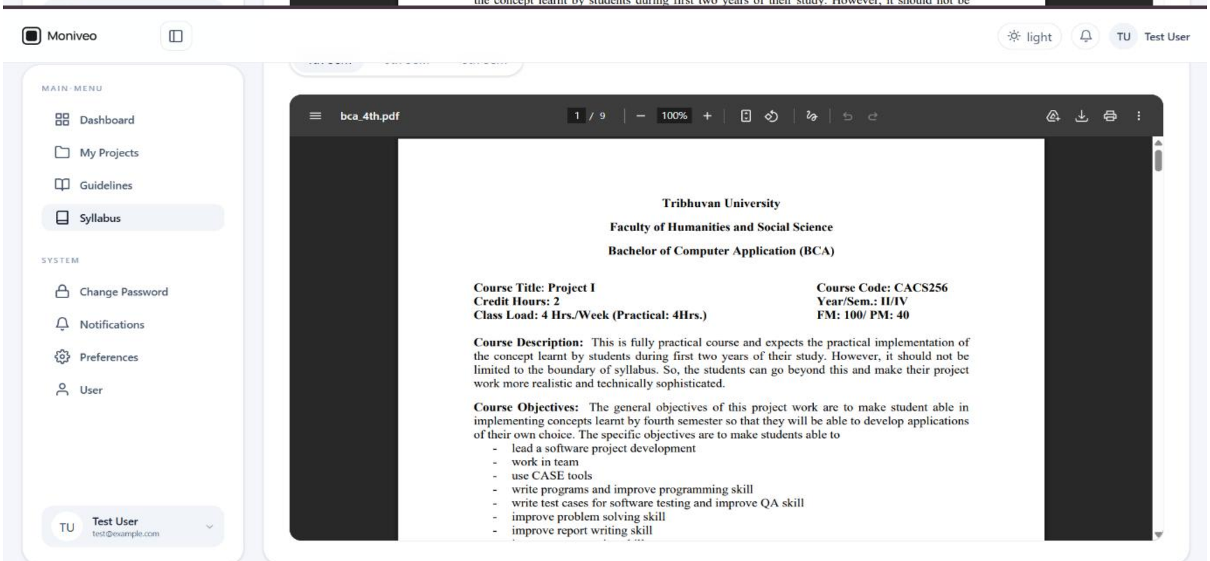


Add project/file:

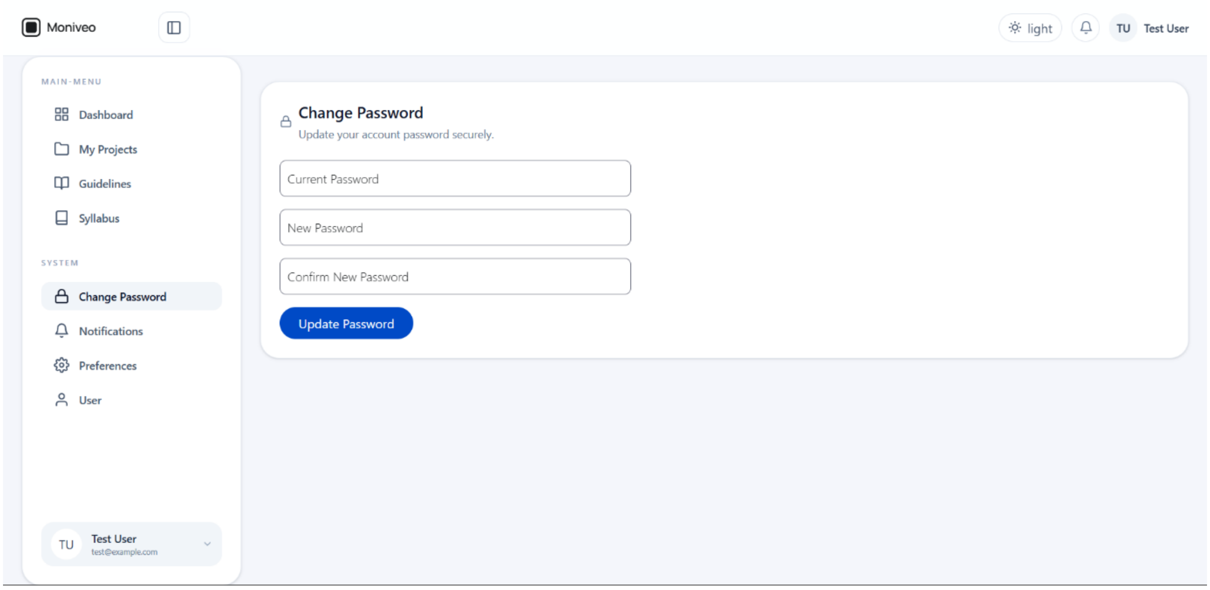


Syllabus:

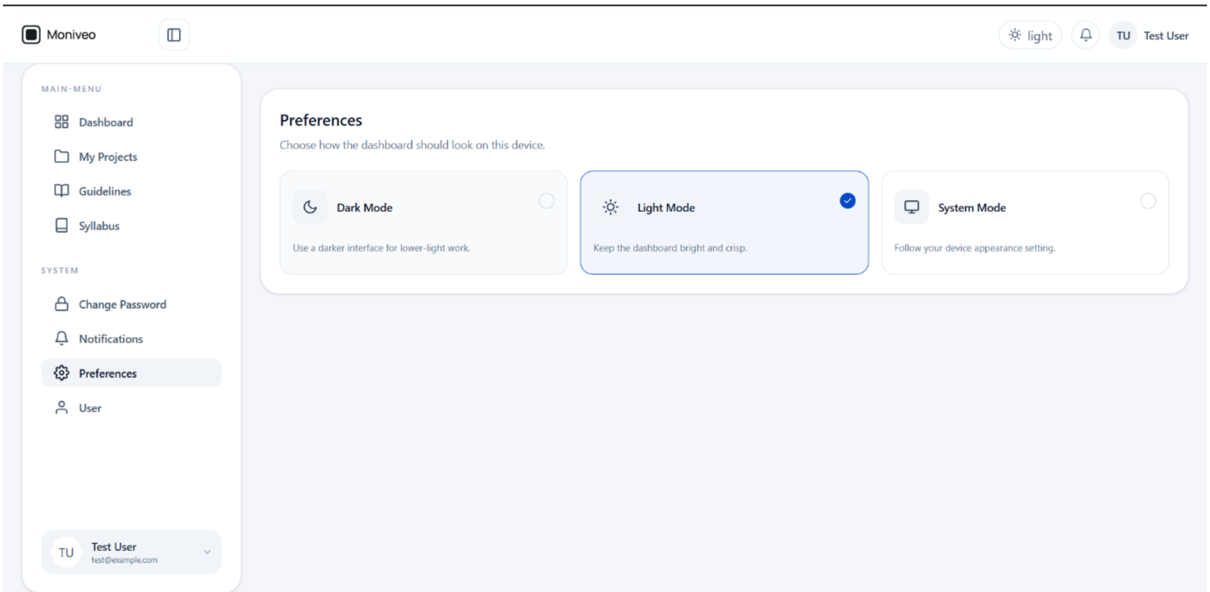




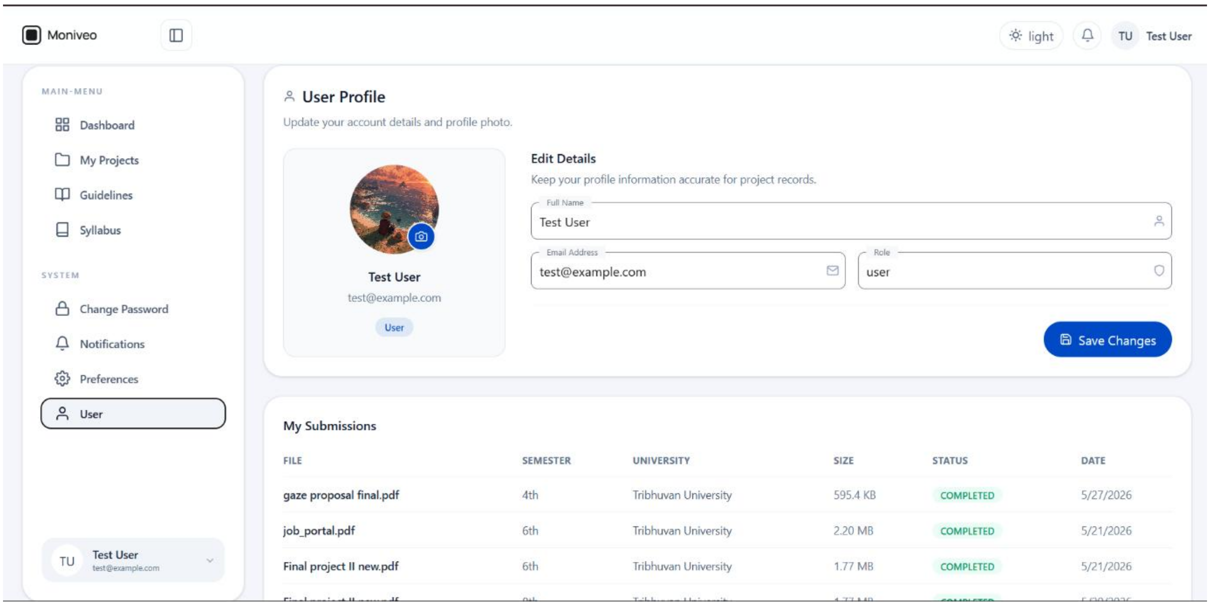
Change Password:



Preference:



User:


Moniveo
light
TU
Test User
MAIN-MENU
Dashboard
My Projects
Guidelines
Syllabus
SYSTEM
Change Password
Notifications
Preferences
User
Test User
test@example.com
User Profile
Update your account details and profile photo.
Test User
test@example.com
User
Edit Details
Keep your profile information accurate for project records.
Full Name
Test User
Email Address
test@example.com
Role
user
Save Changes
My Submissions
FILE
SEMESTER
UNIVERSITY
SIZE
STATUS
DATE
gaze proposal final.pdf
4th
Tribhuvan University
595.4 KB
COMPLETED
5/27/2026
job_portal.pdf
6th
Tribhuvan University
2.20 MB
COMPLETED
5/21/2026
Final project II new.pdf
6th
Tribhuvan University
1.77 MB
COMPLETED
5/21/2026